

研究型进阶专题 A · 第 2 讲

真实 Benchmark 重复实验：随机性、Failure Stability 与 pass^k

更新时间：2026-09-29

● **本讲只做一件事：**把“我跑过一个 Benchmark”升级为“我知道同一个 Task 的成功/失败到底稳不稳定”。继续固定 τ^3 -bench v1.0.1，不换框架。

1. 为什么一次成功或一次失败都不够

LLM Agent 是随机系统，真实 API、用户模拟器、消息生成和运行时都可能引入非确定性。一次 $\text{reward}=1$ 只能说明“这一条 trajectory 成功”，不能说明这个系统对该 task 可靠；一次 $\text{reward}=0$ 也不能直接说明它稳定失败。

重复实验矩阵：同一个 Task 必须看多次 Trial

	Trial 1	Trial 2	Trial 3	Trial 4
Task 1	PASS	PASS	PASS	PASS
Task 2	PASS	PASS	FAIL	PASS
Task 3	PASS	FAIL	PASS	FAIL
Task 4	FAIL	FAIL	FAIL	FAIL
Task 5	PASS	PASS	INFRA	PASS

稳定通过 $\neq$ 偶然通过；稳定失败 $\neq$ 一次失败。先重复，再解释。

图 1 固定 Task 后做重复 Trial，才能看出稳定性结构

● **通俗比喻：**一次考试考 100 分，不等于每次都能 100 分。Agent Reliability 更像“换一套同等条件再考，结果是否仍然稳定”。

● **科研坑：**如果只挑一次成功 trajectory 当案例，很容易出现 cherry-picking（挑结果）。如果只挑一次失败设计“修复方法”，也可能是在修一个随机噪声。

2. Trial、Seed 与 Nondeterminism 到底是什么

概念	英文全称/中文	在 τ^3 -bench 中的作用
Trial	Repeated Trial / 重复试验	同一个 task 的一次独立 simulation。`--num-trials 4` 会给每个 task 做 4 次。

Seed	Random Seed / 随机种子	官方 batch runner 用 base seed 生成每个 trial 的 seed；固定 base seed 可以固定 trial seed schedule。
Nondeterminism	非确定性	即使配置相同，外部模型 API 仍可能产生不同输出；seed 不是“百分百复现按钮”。
Trajectory	轨迹	每个 trial 的消息、tool call、tool result、终止与 reward 记录。

● **底层原理：** τ^3 -bench v1.0.1 的 batch runner 会先 `random.seed(config.seed)`，再为 `num_trials` 生成 trial seeds。也就是说，base seed 控制“重复实验的种子表”；但模型服务端实现可能仍有不可控随机性。

3. pass^k ：为什么它比单次 Success Rate 更严格

官方 `agent_metrics.py` 定义：对一个 task，共 n 次可评估 trial，其中 c 次成功，

$$\text{pass}^k = C(c, k) / C(n, k)$$

它表示：从这 n 次 trial 中不放回地抽 k 次，抽到的 k 次全部成功的比例。注意它与代码生成常见的 $\text{pass}@k$ （“ k 次里至少一次成功”）不是一个方向。这里 k 越大，要求越严格。

4 trials 中成功次数	pass^1	pass^2	pass^3	pass^4	直观含义
4/4	1.00	1.00	1.00	1.00	稳定通过
3/4	0.75	0.50	0.25	0.00	看似不错，但无法保证 4 次全过
2/4	0.50	0.167	0	0	明显不稳定
0/4	0	0	0	0	稳定失败候选

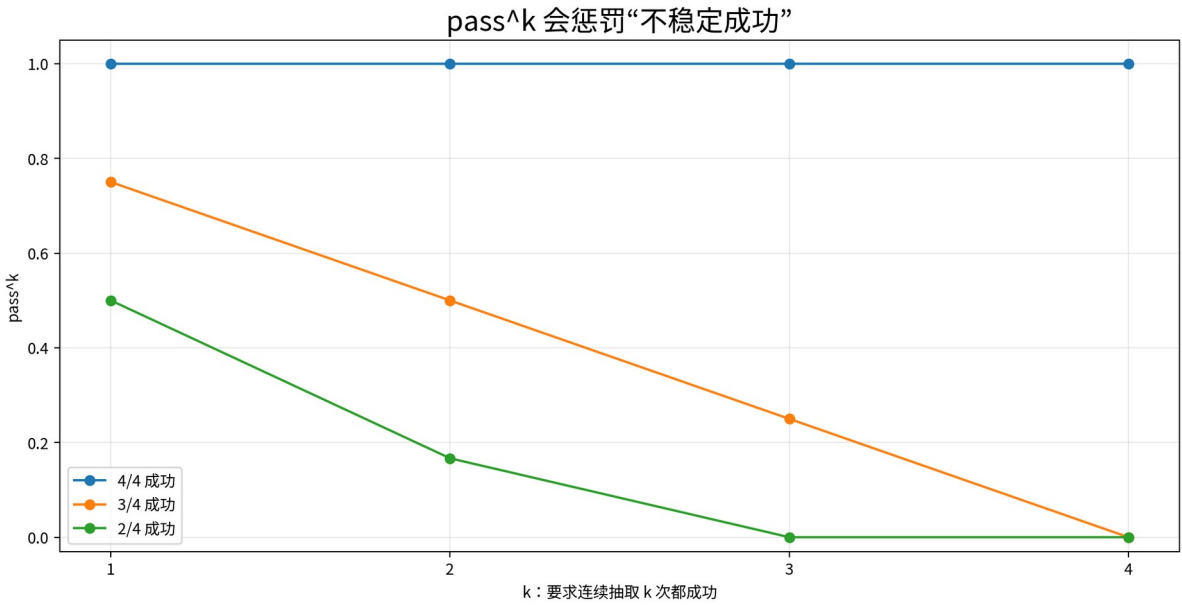


图 2 pass^k 对“不稳定成功”非常敏感
Agent Systems 研究型进阶专题 A · 第 2 讲

● **常见误区：**不要把 pass^4 理解成“4 次里成功至少 1 次”。恰好相反：它要求抽出的 4 次都成功；当 $n=4$ 时，就是 $4/4$ 才为 1。

4. infrastructure_error 为什么必须单独处理

真实 API 实验里，429、断连、服务超时、Harness 进程异常等可能让 simulation 根本没有正常完成。官方 metrics 会把 `infrastructure_error` 从能力指标中排除，同时单独统计数量。

● **关键边界：**“模型做错了任务”与“这次任务根本没有被正常执行完”是两种不同事件。把两者混成 FAIL，会污染能力估计；把 INFRA 全部删掉又不报告，会掩盖系统工程问题。正确做法是：能力指标排除，系统可靠性指标单列。

5. 本讲的 Task-level 稳定性分类

Task-level 稳定性分类：先看模式，再读 Trace



基础设施错误（INFRA）单独记：它既不是 PASS，也不应草率计成 Model FAIL。

图 3 先按跨 trial 模式筛选，再决定读哪些 Trace

类别	本讲操作定义	研究意义
Stable Pass	全部可评估 trial 成功	说明该 task 在当前配置下暂未暴露失败；不代表普遍可靠。
Stochastic	有成功也有失败	最值得研究随机性、边界条件、状态分叉与 error correlation。
Stable Fail	全部可评估 trial 失败	优先检查 First Bad Span 是否重复；重复才可能形成机制性现象。
Infra-only / Infra-heavy	大量 infrastructure_error	先修运行环境，不做模型能力结论。

● **不是论文分类法：**Stable Pass / Stochastic / Stable Fail 是本讲用于 pilot 的操作性标签，不是自然界永恒类别。正式论文要给出样本数、置信区间、任务分布和边界。

6. 5 tasks × 4 trials：为什么是本讲的最低闭环

20 条 trajectory 对个人研究者仍然便宜、可人工审查，而且刚好能计算 pass^1 到 pass^4 。它不是正式论文样本量，只是“发现现象”的 pilot。

设计项	固定值	原因
Benchmark	τ^3 -bench v1.0.1	避免 task/evaluator 漂移
Domain	retail	延续第 1 讲，减少新变量
Tasks	同一次 run 中固定 5 个	先看 task-level 模式
Trials	4	能直接观察 $\text{pass}^1 \dots \text{pass}^4$
Concurrency	1	先减少 rate-limit / 并发噪声
Base seed	300	固定 trial seed schedule
Trace	verbose logs = all	保留每次 LLM 调用证据

7. 怎样从“随机失败”走向 Research Observation

从重复 Benchmark 到 Research Observation



图 4 重复实验只是起点，Trace 机制证据才是后续

1. 先跑 5×4 ，得到 task × trial 成败矩阵。
2. 筛出 Stochastic / Stable Fail task。
3. 逐 trial 标 First Bad Span，不要先看最终失败后果。
4. 比较同一 task 的失败 trial：最早错误是否同类、是否同一工具、是否出现在相似步数。
5. 只有模式重复出现，才写 Observation；否则标记为“暂不稳定”。
6. 提出 Hypothesis 时同时写 Falsifier，再设计只改变一个变量的 Controlled Intervention。

● **例子：**如果 Task 12 四次中两次成功、两次失败，而失败两次都在同一读取工具之后把同一状态字段理解反了，这比“Task 12 成功率 50%”更接近真正可研究的机制现象。

8. 本讲必须保存的证据

- 原始 `results.json`，不要手工改。
- 每条 episode 的 reward、trial seed、termination、cost、duration、token。
- task_stability.csv：每个 task 的成功次数和 pass^k 。

- failure_matrix.csv: 跨 trial 一眼看模式。
- Trace review queue: 只优先审 Stochastic / Stable Fail。
- First Bad Span 人工标注表。
- Run Manifest: benchmark commit、模型/provider、完整命令、结果 SHA256。

● **最低闭环能力:** 跑出 20 条 trajectory 不是目标。你真正要会的是: 知道哪些失败稳定、哪些失败随机、哪些其实是基础设施; 然后能从同一 task 的多条 Trace 中提出一个可证伪的观察。

9. 本讲不做什么

- 不换第二个模型做排行榜式比较。
- 不在 5 个 task 上做显著性检验并声称论文结论。
- 不因为发现一个 failure 就立刻设计新 Agent 模块。
- 不把模型 API 的一次 429 算成 reasoning failure。
- 不把 pass^k 当成独立于 task 分布的“模型固有属性”。

10. 做完本讲你应当能回答

问题	你应当掌握的答案
为什么要重复 trial?	区分稳定行为、随机分叉和基础设施异常。
seed 能保证完全复现吗?	不能; 它固定框架随机源的一部分, 不等于固定远端模型实现。
pass ^k 在测什么?	同一 task 多次尝试都成功的可靠性, k 越大越严格。
先读哪些 Trace?	Stochastic 与 Stable Fail, 尤其失败模式跨 trial 重复的 task。
什么时候能写 Research Question?	Observation 在重复 trial 中稳定后, 再做文献检索与控制干预。